Interview Prep Guidelines 3

Part 4 (Questions 61–80) – TypeScript (Senior Front-End Interview)

---

61. What is TypeScript?

Answer

TypeScript is a **superset of JavaScript** that adds **static typing**. It compiles to plain JavaScript.

Benefits:

- Catches errors at compile time
- Better IntelliSense
- Easier refactoring
- Better maintainability
- Improved developer productivity

---

62. Why use TypeScript?

Instead of finding errors at runtime:

```
const age = "25";

console.log(age.toFixed(2));
```

Runtime Error ✖

TypeScript catches it before running.

```
const age: string = "25";

age.toFixed(2);
```

Compile Error ✔

---

63. Difference between `interface` and `type`

Interface

```
interface User {
    id: number;
    name: string;
}
```

Type

```
type User = {
    id: number;
    name: string;
};
```

Differences

Interface	Type
Can merge declarations	Cannot
Better for object shapes	More flexible
Can extend interfaces	Can use unions/intersections

Interview Answer:

> *I generally use **interfaces** for object contracts and **type aliases** for unions, intersections, mapped types, and utility types.*

---

64. What are Generics?

Generics make functions reusable for multiple data types.

Without generics:

```
function getNumber(value: number) {
    return value;
}
```

With generics:

```
function identity<T>(value: T): T {
    return value;
}

identity(10);

identity("Hello");

identity(true);
```

`T` represents any type.

---

65. What is `keyof`?

`keyof` returns the keys of an object as a union.

```
interface User {
    id: number;
    name: string;
}

type Keys = keyof User;
```

Result:

```
"id" | "name"
```

Useful for creating type-safe property access.

---

66. Explain this function

```
function getValue<T>(obj: T, key: keyof T) {
    return obj[key];
}
```

Usage:

```
const user = {
    name: "John",
    age: 30
};

getValue(user, "name");
```

Works.

```
getValue(user, "salary");
```

Compile Error.

Because `"salary"` is not a key of `user`.

---

67. What is `extends` in Generics?

It restricts generic types.

```
function printLength<T extends { length: number }>(value: T) {
    console.log(value.length);
}
```

Works:

```
printLength("Hello");

printLength([1,2,3]);
```

Fails:

```
printLength(10);
```

Because numbers don't have a `length` property.

---

68. What is `Record`?

Creates an object type.

```
type Users = Record<string, number>;
```

Equivalent:

```
{
    [key: string]: number;
}
```

Example:

```
const scores: Record<string, number> = {
    John: 95,
    Jane: 88
};
```

---

69. What is `Partial`?

Makes every property optional.

```
interface User {
    name: string;
    age: number;
}
```

```
Partial<User>
```

Becomes:

```
{
    name?: string;
    age?: number;
}
```

Useful for update APIs.

---

70. What is `Required`?

Makes every property required.

```
interface User {
    name?: string;
}
```

After:

```
Required<User>
```

Result:

```
{
    name: string;
}
```

---

71. What is `Readonly`?

Prevents modification.

```ts
interface User {
    name: string;
}

const user: Readonly<User> = {
    name: "John"
};

user.name = "Mike";
```

Compile Error.

---

72. What is `Pick`?

Selects only specific properties.

```ts
interface User {
    id: number;
    name: string;
    age: number;
}
```

```ts
type UserPreview = Pick<User, "id" | "name">;
```

Result:

```ts
{
    id: number;
    name: string;
}
```

---

73. What is `Omit`?

Removes properties.

```ts
type UserWithoutAge = Omit<User, "age">;
```

Result:

```ts
{
    id: number;
    name: string;
}
```

---

74. Union vs Intersection

Union

```ts
type A = string | number;
```

Can be either.

Intersection

```ts
type A = {
    name: string;
};

type B = {
    age: number;
};

type C = A & B;
```

Result:

```ts
{
    name: string;
    age: number;
}
```

Combines both types.

---

75. What is Type Narrowing?

TypeScript reduces a broader type to a more specific one.

```ts
function print(value: string | number) {

    if (typeof value === "string") {
        console.log(value.toUpperCase());
    }

}
```

Inside the `if`, TypeScript knows `value` is a string.

---

76. What is a Type Guard?

A type guard narrows types.

Example:

```ts
if (typeof value === "string") {
}
```

Or

```ts
if ("name" in user) {
}
```

Or custom:

```ts
function isDog(animal: Dog | Cat): animal is Dog {
    return "bark" in animal;
}
```

---

77. What is `unknown` vs `any`?

any

Turns off type checking.

```
let value: any;

value.test();

value.foo.bar();
```

No errors.

unknown

Safer.

```
let value: unknown;
```

Must check type first.

```
if (typeof value === "string") {
    console.log(value.toUpperCase());
}
```

Prefer `unknown` over `any`.

---

78. What is `never`?

Represents values that never occur.

Example:

```
function error() {
    throw new Error();
}
```

Return type:

```
never
```

Also used for exhaustive switch checks.

---

79. Enum vs Union

Enum:

```
enum Role {
    Admin,
    User
}
```

Preferred modern approach:

```
type Role = "Admin" | "User";
```

Union types are often simpler, tree-shakable, and more flexible.

---

80. What are the biggest TypeScript best practices?

1. Avoid `any`

Prefer:

```
unknown
```

or proper interfaces.

2. Use Generics

Instead of duplication.

3. Prefer Interfaces

For objects.

4. Use Utility Types

- Pick
- Omit
- Partial
- Required
- Readonly

5. Enable Strict Mode

```
{
  "strict": true
}
```

6. Avoid Type Assertions

Bad:

```
const user = {} as User;
```

Better:

Initialize properly.

7. Write Reusable Types

Avoid repeating interfaces.

☆ Senior Interview Follow-up Questions

Why use Generics?

To create reusable, type-safe code without duplicating logic.

Why is `unknown` safer than `any`?

`unknown` forces you to validate the type before using it, preventing many runtime errors.

Why use `keyof`?

It ensures you only access valid object properties at compile time.

Interface or Type?

Recommended answer:

> *I use **interfaces** for object definitions and API contracts because they're extendable and support declaration merging. I use **type aliases** for unions, intersections, mapped types, conditional types, and utility types.*

What TypeScript features do you use most in React?

A strong senior answer:

- Interfaces for Props
- Generics for reusable hooks/components
- Utility Types (`Partial`, `Pick`, `Omit`)
- `keyof`
- `Record`
- Type Guards
- Discriminated Unions
- Strict Mode
- Typed API responses
- Generic custom hooks

These are the TypeScript concepts most frequently tested in senior frontend interviews. The next section would typically cover **Performance, React optimization, Web Vitals, architecture, state management, and debugging**, which are common discussion topics for senior-level roles.